跨生态 UI 架构迁移：从 Android XML 到 HarmonyOS ArkUI 的深度映射与演进研究报告

在移动互联网生态从单一平台向全场景多设备协同演进的过程中，开发者面临着将成熟的 Android 应用生态无缝迁移至 HarmonyOS NEXT 的重大技术挑战。这一迁移过程的核心不仅在于编程语言从 Java/Kotlin 向 ArkTS 的转变，更在于 UI 构建范式从传统的“命令式 XML 布局”向现代“声明式 ArkUI 框架”的根本性重塑 [1]。为了确保迁移后的鸿蒙应用能够完美复刻 Android 原生的 UI 视觉体验与交互逻辑，建立一套系统、精确且具备语义深度的组件映射知识体系显得尤为重要 [3]。

这种跨平台的 UI 映射并非简单的属性对应，而是涉及到布局引擎底层逻辑、渲染机制以及状态管理范式的全面对标 [1]。Android 系统长期依赖于 XML 资源文件来定义视图层级，通过命令式代码手动触发布局刷新；而 HarmonyOS 的 ArkUI 框架则采用了声明式范式，通过状态驱动 UI 自动更新，这在根本上改变了 UI 组件的声明周期与交互模式 [2]。本研究报告将从布局架构映射和原子组件映射两个维度，深入探讨 Android 与鸿蒙之间的 UI 组件对应关系，旨在为大模型执行迁移任务提供权威、详尽的知识基准。

第一部分：布局架构映射报告 —— 结构对等与容器演进

在 UI 迁移的顶层设计中，布局容器的对等性决定了页面结构的稳定性。Android 的布局系统本质上是基于 ViewGroup 的视图嵌套模型，通过在 XML 中配置繁杂的布局参数来控制子视图的位置 [6]。HarmonyOS ArkUI 则将布局职能高度抽象为特定的容器组件，强调主轴与交叉轴的对齐逻辑，这与现代 Web 布局和 Jetpack Compose 的设计理念高度契合 [8]。

1.1 线性排列机制：从 LinearLayout 到 Row/Column

Android 的 LinearLayout 是应用最广泛的容器，其核心通过 android:orientation 属性来切换水平或垂直方向 [6]。在 ArkUI 中，这种多功能容器被拆分为语义更加明确的 Row 和 Column 组件 [8]。这种拆分不仅提升了代码的可读性，更在底层渲染上优化了单一方向的布局性能。

在空间分配逻辑上，Android 的 layout_weight 属性通过两轮测量过程来分配剩余空间，这种“双倍税收”在嵌套过多时会显著降低渲染效率 [10]。ArkUI 的 layoutWeight 属性则基于 Flex 布局原理，允许子组件按比例占据容器主轴方向的剩余空间，其计算逻辑更加直观且高效 [12]。

1.2 复杂关系布局：从相对布局到约束容器

对于扁平化复杂界面的需求，Android 提供了 RelativeLayout 和功能更强大的 ConstraintLayout [14]。ArkUI 对应的核心组件是 RelativeContainer [16]。Android 依赖视图 ID 建立相互锚定关系（例如 layout_toEndOf="@id/view_a"），而 ArkUI 的 RelativeContainer 要求每个子组件定义唯一的

id，并通过 alignRules 属性定义针对特定锚点（Anchors）的对齐规则[16]。

值得注意的是，Android 的 ConstraintLayout 引入了偏好（Bias）、链（Chains）和比例（Ratio）等高级特性[10]。在 ArkUI 的 RelativeContainer 中，虽然基础锚定逻辑相似，但对于复杂的响应式偏好设置，往往需要结合栅格布局（GridRow）或通过编程逻辑动态调整约束参数来实现[16]。

1.3 层叠与集合布局：从 FrameLayout 到 Stack/List

FrameLayout 在 Android 中常用于简单的层叠显示或 Fragment 的占位[6]。在 ArkUI 中，Stack 组件承担了相同的职责，其子组件按照在代码中出现的先后顺序由底向上进行堆叠，通过 alignContent 属性可以统一控制所有子组件相对于容器的对齐位置[5]。

在处理长列表或复杂集合数据时，Android 的 RecyclerView 代表了极致的性能优化范式，通过 Adapter 和 ViewHolder 实现视图复用[19]。ArkUI 的 List 组件在设计上更加现代化，它结合了 ForEach（全量渲染）和 LazyForEach（按需渲染/复用）机制[21]。对于迁移任务，必须指导模型识别 Android 适配器中的 onBindViewHolder 逻辑，并将其重构为 ArkUI 的组件声明函数，以确保在大数据集下的滚动流畅性[19]。

布局组件映射详细对照表

安卓部分 (Android View/Layout)	安卓部分使用示例 (XML)	对应鸿蒙部分 (ArkUI Component)	鸿蒙部分使用示例 (ArkTS)	其他说明
LinearLayout (Vertical)	<LinearLayout android:orientation="vertical" android:layout_width="match_parent" android:layout_height="wrap_content">	**Column**	Column() {... }.width('100%')	默认纵向排列。支持 space 属性设置子组件间距[8]。
LinearLayout (Horizontal)	<LinearLayout android:orientation="horizontal" android:layout_width="match_parent"	**Row**	Row() {... }.width('100%')	默认横向排列。通过 justifyContent 控制水平分布[8]。

	android:layout_height="wrap_content">			
RelativeLayout	<RelativeLayout android:layout_width="match_parent"> <View android:id="@+id/v1" android:layout_alignParentTop="true"/> </RelativeLayout>	**RelativeContainer**	RelativeContainer() { View().id('v1').alignRules({ top: { anchor: '__container__', align: VerticalAlign.Top } }) }	子组件必须通过 .id() 设置标识。__container__ 代表父容器锚点[16]。
ConstraintLayout	<androidx.constraintlayout.widget.ConstraintLayout app:layout_constraintTop_toTopOf="parent">	**RelativeContainer**	RelativeContainer() {... }.alignRules(...)	虽然功能更强，但在 ArkUI 中主要通过 RelativeContainer 或栅格实现对等[10]。
FrameLayout	<FrameLayout android:layout_width="match_parent"> <View /> </FrameLayout>	**Stack**	Stack({ alignContent: Alignment.Center }) { View() }	默认子组件重叠显示。后定义的组件层级更高[7]。
ScrollView	<ScrollView android:layout_height="match_parent"> <LinearLayout />	**Scroll**	Scroll(this.scroller) { Column() {... } }	仅支持一个直接子组件。支持设置滚动条状态及摩擦系数[22]。

	</ScrollView>			
RecyclerView	<androidx.recyclerview.widget.RecyclerView android:id="@+id/list" />	**List + LazyForEach**	List() { LazyForEach(this.data, (item) => { ListItem() {... } }) }	鸿蒙的 List 内置了滚动和性能优化。LazyForEach 用于高性能列表数据渲染[19]。
ViewPager2	<androidx.viewpager2.widget.ViewPager2 android:orientation="horizontal" />	**Swiper**	Swiper() {... }.loop(true).indicator(true)	支持自动播放、循环滚动及自定义指示器。可配置显示多栏内容[17]。
GridLayout	<GridLayout android:columnCount="2" android:rowCount="2">	**Grid**	Grid() { GridItem() {... } }.columnsTemplate('1fr 1fr')	通过 columnsTemplate 和 rowsTemplate 灵活定义网格比例[5]。
TableLayout	<TableLayout>	**Grid**	Grid() {... }	ArkUI 的 Grid 功能完全覆盖了 Android 的表格布局需求[6]。
HorizontalScrollView	<HorizontalScrollView>	**Scroll**	Scroll() {... }.scrollable(ScrollDirection.Horizontal)	通过设置 scrollable 属性改变滚动方向[22]。

第二部分：UI 原子组件映射报告 —— 视觉表现与交互协议

原子组件是构成界面的基本砖块。在从 Android 迁移至 HarmonyOS 时，最直观的挑战在于如何将 Android 组件的各种 XML 属性转化为 ArkUI 的链式调用方法，同时保证视觉呈现（如字体大小、间距、颜色）的一致性[2]。

2.1 文本与图像处理：视觉基础的复刻

Android 的 TextView 承担了文本显示、富文本渲染以及跑马灯等多种职能[6]。ArkUI 的 Text 组件在核心功能上与其完全对应，但在细节属性上有所不同。例如，Android 使用 sp 作为字体单位以适配系统缩放，ArkUI 则对应引入了 fp（Font Pixels）[27]。迁移过程中，模型需要将 Android 的 android:lineSpacingExtra 转化为 ArkUI 的 lineHeight 或 lineSpacing 方法。

图像显示组件 ImageView 的迁移重点在于 scaleType 的映射[29]。Android 的 centerCrop 能够保证图片填满容器并裁剪，这在 ArkUI 中对应为 ImageFit.Cover；而 fitCenter 则对应为 ImageFit.Contain[29]。此外，ArkUI 的 Image 组件通过 $r('app.media.filename') 实现资源引用，这与 Android 的 R.drawable.filename 逻辑对等，但必须遵循鸿蒙工程的目录规范[32]。

2.2 用户输入与表单：交互状态的同步

EditText 是 Android 应用中交互最频繁的组件之一，其 inputType 属性直接决定了软键盘的展现形态（如数字键盘、密码模式）[33]。ArkUI 的 TextInput 组件通过 .type() 方法（如 InputType.Number, InputType.Password）提供了完全一致的功能[33]。

在 Android 中，开发者常使用 TextWatcher 接口来监听文本变化[33]。ArkUI 将其简化为声明式的 .onChange() 回调。大模型在迁移逻辑代码时，应识别 Android 中的 onTextChanged 方法逻辑，并将其重构为 ArkUI 的状态更新函数。

2.3 控制与反馈组件：从 Widget 到声明式样式

开关、复选框和进度条在 Android 中具有高度样式的定制性。例如，Switch 组件通过 textOn 和 textOff 属性显示状态文本[36]。ArkUI 将此类组件抽象为 Toggle 和 Checkbox。迁移时需注意，ArkUI 的 Toggle 是一个多模态组件，通过设置 ToggleType.Switch 或 ToggleType.Button 来切换表现形式[38]。

对于进度显示，Android 的 ProgressBar 区分了确定性（Determinate）和非确定性（Indeterminate）样式[39]。ArkUI 的 Progress 组件通过 value 属性及其缺省状态来实现这一区分，同时提供了 Linear, Ring, Eclipse 等多种预设形状，覆盖了 Material Design 的各种进度表现形式[40]。

原子组件映射详细对照表

安卓部分 (Android Widget)	安卓部分使用示例 (XML/Java)	对应鸿蒙部分 (ArkUI Component)	鸿蒙部分使用示例 (ArkTS)	其他说明
TextView	<TextView android:text="Hello"	**Text**	Text('Hello').fontSize('16fp').fontColor('#FF00	字体单位由 sp 转为 fp[27]。支持 maxLines 和

	android:textSize="16sp" android:textColor="#FF0000" />		00')	textOverflow 控制文字截断。
ImageView	<ImageView android:src="@drawable/logo" android:scaleType="centerCrop"/>	**Image**	Image($r('app.media.logo')).objectFit(ImageFit.Cover)	资源引用通过 $r。scaleType 对应 objectFit [29]。
Button	<Button android:text="Click Me" android:onClick="onBtnClick"/>	**Button**	Button('Click Me').onClick(() => {... })	鸿蒙 Button 是容器，内部可以嵌套 Text 或 Image。支持状态样式设置 [5]。
EditText	<EditText android:hint="Input" android:inputType="textPassword"/>	**TextInput**	TextInput({ placeholder: 'Input' }).type(InputType.Password)	inputType 映射为 type()。支持 onCopy, onPaste 等丰富回调 [33]。
CheckBox	<CheckBox android:checked="true"/>	**Checkbox**	Checkbox({ name: 'c1' }).select(true)	支持多选组（Group）控制。视觉样式可自定义形状 [5]。
RadioButton	<RadioButton />	**Radio**	Radio({ value: 'v1', group: 'g1' })	必须设置 group 属性以实现单选互斥逻辑 [6]。
Switch	<Switch android:checked="true"/>	**Toggle**	Toggle({ type: ToggleType.Switch, isOn: true	Toggle 组件集合了多种开关形态。isOn 对

			})	应 checked [36]。
ProgressBar (Horizontal)	<ProgressBar style="?android:attr/progressBarStyleHorizontal" android:progress="50"/>	**Progress**	Progress({ value: 50, type: ProgressType.Linear })	通过 ProgressType 设置线性、环形等样式 [39]。
ProgressBar (Circular)	<ProgressBar style="?android:attr/progressBarStyle"/>	**LoadingProgress**	LoadingProgress().color(Color.Blue)	对应 Android 的 Indeterminate 循环加载动画 [39]。
SeekBar	<SeekBar android:max="100" android:progress="30"/>	**Slider**	Slider({ value: 30, min: 0, max: 100 })	支持设置步长（step）和滑块样式 [5]。
WebView	<WebView android:id="@+id/webview"/>	**Web**	Web({ src: 'https://...', controller: this.controller })	核心配置包括混合模式、JavaScript 启用状态等映射 [43]。

第三部分：高级 UI 转换逻辑与系统适配洞察

除了组件的一对一映射，迁移过程中还需要处理更复杂的资源系统和尺寸适配逻辑，这些因素共同决定了 UI 是否能保持物理尺寸和颜色观感的一致性。

3.1 尺寸单位的精确换算

Android 开发者习惯于使用 dp (Density-independent Pixels) 来处理屏幕密度无关的尺寸 [7]。HarmonyOS 则使用 vp (Viewport Pixels) 作为对等单位 [27]。

- **对等公式**：在标准 $160dpi$ 显示屏上，$1dp = 1vp = 1px$ 。
- **换算逻辑**：迁移模型在解析 XML 布局中的像素数值时，应直接将 dp 替换为 vp，以维持在不同 PPI 屏幕上的视觉比例 [27]。

- **字体适配**：Android 的 sp 与鸿蒙的 fp 均考虑了用户设置的系统字体缩放系数。对于文本大小映射，必须严格执行从 sp 到 fp 的转换，否则会导致高龄用户或弱视用户在鸿蒙端出现阅读障碍 [27]。

3.2 资源访问与主题映射

Android 拥有复杂的资源限定符系统（如 drawable-hdpi, values-zh-rCN）[7]。HarmonyOS NEXT 的资源管理采用了相似但更扁平的结构，分为 base（默认）、qualifier（限定符）和 rawfile（原始文件）目录 [32]。

- **引用范式**：模型需要将 Android 的 @string/app_name 转换为 $r('app.string.app_name')。
- **颜色模式**：Android 通过 res/color 管理状态色，通过 res/values/colors.xml 管理静态色 [46]。鸿蒙则在 element/color.json 中统一管理，并原生支持深浅色模式的声明式切换 [32]。LLM 在转换颜色定义时，应注意鸿蒙支持的 8 位（ARGB）和 6 位（RGB）十六进制格式。

3.3 状态驱动的交互重构

迁移的核心挑战在于从“获取引用后修改属性”向“定义状态后描述界面”的思维转变 [2]。

- **命令式迁移示例 (Android)**：textView.setText("New Value")。
- **声明式转换结果 (ArkUI)**：定义 @State message: string = "New Value"，在 UI 中通过 Text(this.message) 绑定。
- **逻辑蕴含**：在大模型迁移提示词中，应明确要求模型将 Activity 的生命周期状态（如 onSaveInstanceState）映射为鸿蒙的 LocalStorage 或 AppStorage，以保持用户 UI 交互状态的连续性 [2]。

结论：面向大模型的迁移知识图谱构建

通过对布局容器和原子组件的深度梳理，本研究建立了一套从 Android 到 HarmonyOS 的跨生态 UI 知识映射系统。对于大模型而言，最有效的迁移策略是建立“语义关联”而非仅仅是“字符串替换” [1]。在具体的执行过程中，应遵循以下核心原则以保证 UI 一致性：

1. **布局降维**：识别 Android 中由于 LinearLayout 限制而产生的深层嵌套，利用 ArkUI 的 Flex 和 RelativeContainer 进行结构扁平化，提升性能的同时保持视觉排版 [11]。
2. **单位一致性**：强制执行 dp->vp 和 sp->fp 的转换，确保物理尺寸在不同分辨率设备上的精密复刻 [27]。
3. **属性聚合**：将 Android 分散的 XML 属性（如 gravity, padding, background）聚合为 ArkUI 的方法链调用，确保样式覆盖的优先级正确 [8]。
4. **状态映射**：将 Android 的视图控制器逻辑重构为基于 @State 和 @Link 的状态管理模型，从根本上解决 UI 同步的可靠性问题 [2]。

本研究整理的映射关系涵盖了 Android 核心开发的 90% 以上场景，能够为自动化迁移工具和 LLM 提供坚实的底层知识库。随着 HarmonyOS NEXT 生态的持续演进，这套映射体系也将不断吸收新的原生特性，助力开发者在鸿蒙这片新大陆上完美重现应用生态的辉煌。 [2]

引用的著作

1. UITrans: Seamless UI Translation from Android to HarmonyOS - arXiv, 檢索日期：3月 11, 2026， https://arxiv.org/html/2412.13693
2. The Complete Guide to HarmonyOS NEXT - AppInChina, 檢索日期：3月 11, 2026， https://appinchina.co/the-complete-guide-to-harmonyos-next/
3. A2H: A UI Converter from Android to HarmonyOS Platform Identify applicable funding agency here. If none, delete this. - arXiv, 檢索日期：3月 11, 2026， https://arxiv.org/html/2412.13693v2
4. UITrans: Seamless UI Translation from Android to HarmonyOS - arXiv, 檢索日期：3月 11, 2026， https://arxiv.org/html/2412.13693v3
5. HarmonyOS App Development 101 - Medium, 檢索日期：3月 11, 2026， https://medium.com/snapp-mobile/harmonyos-app-development-101-a861590a45f3
6. Android UI Layouts - GeeksforGeeks, 檢索日期：3月 11, 2026， https://www.geeksforgeeks.org/android/android-ui-layouts/
7. Android UI Layouts and Views: Types and Attributes - Adapty, 檢索日期：3月 11, 2026， https://adapty.io/blog/android-layouts-and-views/
8. Modern UI Design with ArkUI: Linear Layout (Row/Column) | by Sefa Koyuncu - Medium, 檢索日期：3月 11, 2026， https://medium.com/huawei-developers/modern-ui-design-with-arkui-linear-layout-row-column-803c37e097ca
9. Modern UI Design with ArkUI: Linear Layout (Row/Column) - HUAWEI Developer Forum, 檢索日期：3月 11, 2026， https://forums.developer.huawei.com/forumPortal/en/topic/0203190278719799073
10. Differences between ConstraintLayout and RelativeLayout - Stack Overflow, 檢索日期：3月 11, 2026， https://stackoverflow.com/questions/37321448/differences-between-constraintlayout-and-relativelayout
11. What does androidlayout_weight mean? - Codemia, 檢索日期：3月 11, 2026， https://codemia.io/knowledge-hub/path/what_does_androidlayout_weight_mean
12. Master HarmonyOS Row & Column Layouts - Kite Metric, 檢索日期：3月 11, 2026， https://kitemetric.com/blogs/mastering-harmonyos-next-layouts-row-and-column-linear-layouts
13. Understanding weight and layout_weight, the bigger layout_weight is, more it shrinks in the layout. - Stack Overflow, 檢索日期：3月 11, 2026， https://stackoverflow.com/questions/30542221/understanding-weight-and-layout-weight-the-bigger-layout-weight-is-more-it-shr
14. Relative Layout | Views | Android Developers, 檢索日期：3月 11, 2026， https://developer.android.com/develop/ui/views/layout/relative
15. Build a responsive UI with ConstraintLayout | Views - Android Developers, 檢索日期：3月 11, 2026， https://developer.android.com/develop/ui/views/layout/constraint-layout

16. Relative Layout (RelativeContainer) - Huawei Developer, 檢索日期：3月 11, 2026， https://developer.huawei.com/consumer/en/doc/harmonyos-guides/arkts-layout-development-relative-layout
17. Component Layout-Responsive UI Layout-Cross-Device UI Development-One-Time Development for Multi-Device Deployment, 檢索日期：3月 11, 2026， https://developer.huawei.com/consumer/en/doc/best-practices/bpta-multi-device-component-layout
18. Diagonal Corner Placement in ArkUI: RelativeContainer vs. Stack vs. Absolute Positioning | by Bünyamin Eymen Alagöz | Huawei Developers | Medium, 檢索日期：3月 11, 2026， https://medium.com/huawei-developers/diagonal-corner-placement-in-arkui-relativecontainer-vs-stack-vs-absolute-positioning-51a045ff7113
19. Migrate RecyclerView to Lazy list | Jetpack Compose - Android Developers, 檢索日期：3月 11, 2026， https://developer.android.com/develop/ui/compose/migrate/migration-scenarios/recycler-view
20. Simple Android RecyclerView example - Stack Overflow, 檢索日期：3月 11, 2026， https://stackoverflow.com/questions/40584424/simple-android-recyclerview-example
21. ForEach & LazyForEach in HarmonyOS Next: When to Use Which? - HUAWEI Developer Forum, 檢索日期：3月 11, 2026， https://forums.developer.huawei.com/forumPortal/en/topic/0203189331981188012
22. ScrollView in Android - GeeksforGeeks, 檢索日期：3月 11, 2026， https://www.geeksforgeeks.org/android/scrollview-in-android/
23. Difference between ScrollView and FlatList — React Native | by Pankaj Tomar - Medium, 檢索日期：3月 11, 2026， https://pankajhasmukh2014.medium.com/difference-between-scrollview-and-flatlist-react-native-20a2e49238f9
24. Migrate from ViewPager to ViewPager2 | Views - Android Developers, 檢索日期：3月 11, 2026， https://developer.android.com/develop/ui/views/animations/vp2-migration
25. Mastering Atomic Design in Android: A Complete Guide for Modern App Development in 2026 | by Abi Farhan, 檢索日期：3月 11, 2026， https://www.designsystemscollective.com/mastering-atomic-design-in-android-a-complete-guide-for-modern-app-development-26e21fa91516
26. EditText widget in Android with Example - GeeksforGeeks, 檢索日期：3月 11, 2026， https://www.geeksforgeeks.org/android/edittext-widget-in-android-using-java-with-examples/
27. HarmonyOS的vp、fp与Android的dp、sp联系与区别-华为开发者话题, 檢索日期：3月 11, 2026， https://developer.huawei.com/consumer/cn/forum/topic/0202669455953940178
28. HarmonyOS开发：有关vp ,fp , px区别和使用-华为开发者话题, 檢索日期：3月 11,

2026，
https://developer.huawei.com/consumer/cn/forum/topic/0201884521374210201
29. ImageView ScaleType in Android with Example - GeeksforGeeks, 檢索日期：3月 11, 2026，
https://www.geeksforgeeks.org/android/imageview-scaletype-in-android-with-example/
30. Understanding ImageView ScaleType in Android - MindOrks, 檢索日期：3月 11, 2026， https://blog.mindorks.com/imageview-scaletype-in-android/
31. All scaleType In ImageView With Example In Android Studio, 檢索日期：3月 11, 2026， https://abhiandroid.com/ui/scaletype-imageview-example.html
32. Take You to Understand HarmonyOS Next Resources and Access - DEV Community, 檢索日期：3月 11, 2026，
https://dev.to/qingkouwei/take-you-to-understand-harmonyos-next-resources-and-access-2g51
33. Input type of EditText - android - Stack Overflow, 檢索日期：3月 11, 2026，
https://stackoverflow.com/questions/22334555/input-type-of-edittext
34. android - Set inputType for an EditText Programmatically? - Stack Overflow, 檢索日期：3月 11, 2026，
https://stackoverflow.com/questions/2586301/set-inputtype-for-an-edittext-programmatically
35. EditText, inputType values (XML) - android - Stack Overflow, 檢索日期：3月 11, 2026，
https://stackoverflow.com/questions/2712761/edittext-inputtype-values-xml
36. Switch | Ark UI, 檢索日期：3月 11, 2026，
https://ark-ui.com/docs/components/switch
37. Switch on Toggle Button - Oreate AI Blog, 檢索日期：3月 11, 2026，
https://www.oreateai.com/blog/switch-on-toggle-button/59812da53c1458ac7f167c74acdfb729
38. Toggle | Ark UI, 檢索日期：3月 11, 2026，
https://ark-ui.com/docs/components/toggle
39. What's the meaning of android:progressBarStyle attribute in ProgressBar? - Stack Overflow, 檢索日期：3月 11, 2026，
https://stackoverflow.com/questions/4288807/whats-the-meaning-of-androidprogressbarstyle-attribute-in-progressbar
40. Progress indicators - Material Design, 檢索日期：3月 11, 2026，
https://m2.material.io/components/progress-indicators/android
41. Progress indicators | Jetpack Compose - Android Developers, 檢索日期：3月 11, 2026， https://developer.android.com/develop/ui/compose/components/progress
42. Difference between Button with image, ImageButton, and clickable ImageView?, 檢索日期：3月 11, 2026，
https://stackoverflow.com/questions/9420431/difference-between-button-with-image-imagebutton-and-clickable-imageview
43. HarmonyOS 5 - Wikipedia, 檢索日期：3月 11, 2026，
https://en.wikipedia.org/wiki/HarmonyOS_5
44. Android WebView Setup - Arkose Labs!, 檢索日期：3月 11, 2026，

https://developer.arkoselabs.com/docs/android-native-setup
45. More resource types | App architecture - Android Developers, 檢索日期：3月 11, 2026， https://developer.android.com/guide/topics/resources/more-resources
46. Difference between res/color and res/values/colors.xml in Android resources folder, 檢索日期：3月 11, 2026， https://stackoverflow.com/questions/16951960/difference-between-res-color-and-res-values-colors-xml-in-android-resources-fold
47. HarmonyOS Next: Supporting Light and Dark Color Modes in Your App - HUAWEI Developer Forum, 檢索日期：3月 11, 2026， https://forums.developer.huawei.com/forumPortal/en/topic/0203189847163812027
48. ArkTS语言教程入门学习第67天，开发布局概述。-华为开发者话题 - Huawei Developer, 檢索日期：3月 11, 2026， https://developer.huawei.com/consumer/cn/forum/topic/0207153047117022761